Pilots, not passengers – reconstructible reporting in evaluation

Chapter contents.

5 Sep 2026

For evaluators, commissioners and anyone else working out what evaluation analysis should look like now that AI can do much of it.

In our experience, a frontier model such as Claude Opus 5.5, given the terms of reference and the documents from an evaluation, can now answer the desk-based part of most evaluation questions in minutes, about as well and as accurately as a competent evaluator. The models get better every month. How, then, do evaluators and the people they work with remain the pilots? Who chooses the method, who decides what counts as good, and how can anybody else check what was done? The politics and ethics matter, and so do guidelines. This chapter is about methods and code you can run.

Work in progress. None of what follows has yet been used on a live evaluation. The pages are drafts, published early so people can argue with them.

What we have so far comes in four parts, each up for discussion.

A principle: reconstructible reporting

Any evaluation report based on an analysis of text could come with a bundle holding the data and the instructions somebody else would need to rebuild the analysis and reach comparable, though probably not identical, results. The principle is familiar. What is new is that a frontier AI can now do the rebuilding with little effort, so a commissioner no longer has to take the method on trust. The principle applies to analysis done by people, or by people working with AI, as much as to analysis done by AI alone. So far we have said too little about the participatory stages of an evaluation, and about the drafts of a report that go back and forth with stakeholders.

Principles for reconstructible analysis in the age of AI sets out the rules we work to under this principle: who decides what counts as good, why every claim points at a quotation, what a finding is compared against, and what to say about what was not read.

An open format for the bundle

In An open format for reconstructible reporting we propose a published specification for that bundle. An analysis is built from a few steps that anyone can audit, whether people or AI carry them out: choose documents, mark passages, group them into themes, count, judge against a rubric, write. Fuller approaches such as contribution analysis can be put together from the same steps. The format is at version 0.3. Nobody needs to agree on one format, and we expect several overlapping ones to appear and change. In Is there already a standard? we report what we found when we looked for an existing one.

Rubicon for Claude: a free, open-source plugin

Rubicon
EVALUATIVE WORKFLOWS

Rubicon for Claude is a plugin that makes your own Claude work to a more explicit method that somebody else can rebuild. Claude is left to reason however it likes, then has to write down the workflow it followed, and the report is rebuilt from that workflow: every count is made by code, every quotation is found word for word in its document, and a zip goes with the report from which somebody else can recreate it. The aim is a Claude as adaptable as the plain one, but more accurate and open to audit; in our tests so far it is more accurate than plain Claude. It costs nothing beyond a Claude subscription. Your documents and what Claude writes about them stay on your computer, apart from going through Anthropic as usual, and never reach us; the plugin does send us usage telemetry, which you can opt out of. The code is on GitHub under the MIT licence. An MCP server offering the same tools to other assistants is in development. How to install it, and example reports, are on the Get page.

The Rubicon app

A web app for the same job, still in development, with more support for the evaluator. You bring a question about a project's documents to Ruby, the app's assistant, who settles the question with you and proposes a plan in plain words, with its cost, before any analysis runs. When you agree, she carries it out, and you can trace every figure in the result to a passage in a document. The app reads Causal Map projects and uses the Causal Map login. It is not finished, not priced and not yet generally available.

Worked examples

Working papers that put Rubicon to particular evaluation methods, some with runs to report. Rubicon is changing fast, so treat these as a historical record: each describes the version it was run on, not Rubicon as it is now.

- Theory-based evaluation: what commissioners want when they ask for a theory-based evaluation, and a run in which we attached the evidence to each link of a theory of change.
- Contribution analysis: a programme's own theory of change put link by link to a select committee's evidence, with a blind check of what the write-up got wrong.
- Testing rival theories over a corpus: five published explanations of loneliness, and a design for asking which of them fifty interviews support.
- Generating realist mechanisms: generating mechanisms from a corpus, with a control for the ones anybody could have written without it.
- Outcome harvesting: which parts of an outcome harvest a machine should touch at all, and a draft outcomes table for a workshop to argue with.

In the realist paper we generate candidate explanations. In the theory-fit paper we test competing explanations on material that played no part in producing them, and in the contribution analysis paper we test a theory somebody has already committed to in writing. All three are kinds of theory-based evaluation. Outcome harvesting is different, because most of that method is a conversation between people.

How this relates to causal mapping

Rubicon is a different thing from causal mapping and does not replace it. The Rubicon FAQ covers how the two products relate; here we compare the methods.

- Causal mapping builds one model of what people say causes what. See Minimalist coding for causal mapping for the coding style and Task 3 — Analysing data, Answering questions for what you do with the result.
- Rubicon answers one stated question at a time, with a standard fixed in advance. A project holds as many codings as it has questions.
- Both depend on checking the evidence. Quality assurance at each step of the causal coding workflow sets out how we check causal coding, and Ways to causal inference places process tracing and its neighbours among the other routes to a causal claim.
- A simple measure of the goodness of fit of a causal theory to a text corpus measures fit differently, by asking how much of a coded map a theory's own vocabulary can express. That measure favours a theory that covers a lot of the map, while the diagnostic tests in the theory-fit paper favour one that tells explanations apart, so we would run both and report where they disagree.
- Causal mapping differs from related approaches - epistemic, less predictive, unsophisticated, many links, many sources, unclear boundaries explains why causal mapping takes many links from many sources where process tracing takes few links very seriously. With Rubicon we are trying to give many links from many sources some of that scrutiny.

We would like to work with people on this. If this is your field, tell us where the argument is wrong, or where it is worth building on, while these pages are still drafts. Steve Powell, Causal Map Ltd, Bath: hello@causalmap.app.

This page was drafted with the assistance of Claude, an AI model from Anthropic; the arguments and judgements are our own.

PAGES IN THIS CHAPTER

- An open format for reconstructible reporting
- Principles for reconstructible analysis in the age of AI
- Is there already a standard ?
- The jury test
- Theory-based evaluation
- Testing rival theories
- Checking the claims in an AI report cannot find its omissions
- Realist mechanisms
- Contribution analysis
- Outcome harvesting